

University of Asia Pacific

Department of CSE

Assignment On

SJF Comparison with FCFS

(CPU Scheduling Algorithms)

Course Title: Operating System Lab

Course Code: CSE 402

Submitted To

Atia Rahman Orthi

Lecturer

Department of Computer Science
and Engineering

University of Asia Pacific

Submitted By

Sultan Ruhul Kuddus Nayan

ID: 23101159

2022-2023 Batch

Department of CSE

University of Asia Pacific

Date of Submission: 10 August 2026

Code implementation

```
processes = [
    ["P1", 3, 3],
    ["P2", 2, 5],
    ["P3", 5, 4],
    ["P4", 1, 3],
    ["P5", 6, 2]
]

print("== SJF ==\n")

ready = processes[:]
time = 0
sjf_result = []

while ready:

    available = []

    for p in ready:
        if p[1] <= time:
            available.append(p)

    if len(available) == 0:
        time = min(ready, key=lambda x: x[1])[1]
        continue

    job = min(available, key=lambda x: x[2])
    ready.remove(job)

    name, at, bt = job

    ct = time + bt
    tat = ct - at
    wt = tat - bt

    sjf_result.append([name, at, bt, ct, tat, wt])

    time = ct

print("Process\tAT\tBT\tCT\tTAT\tWT")
for i in sjf_result:
    print(i[0], "\t", i[1], "\t", i[2], "\t", i[3], "\t", i[4], "\t",
          i[5])

sjf_avg_tat = sum(i[4] for i in sjf_result) / len(sjf_result)
sjf_avg_wt = sum(i[5] for i in sjf_result) / len(sjf_result)

print("\nExecution Sequence:")
for i in sjf_result:
    print(i[0], end=" ")

print("\n\nAverage TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)
```

```

print("\n\n=== FCFS ===\n")

fcfs = sorted(processes, key=lambda x: x[1])

time = 0
fcfs_result = []

for p in fcfs:

    name, at, bt = p

    if time < at:
        time = at

    ct = time + bt
    tat = ct - at
    wt = tat - bt

    fcfs_result.append([name, at, bt, ct, tat, wt])

    time = ct

print("Process\tAT\tBT\tCT\tTAT\tWT")
for i in fcfs_result:
    print(i[0], "\t", i[1], "\t", i[2], "\t", i[3], "\t", i[4], "\t",
i[5])

fcfs_avg_tat = sum(i[4] for i in fcfs_result) / len(fcfs_result)
fcfs_avg_wt = sum(i[5] for i in fcfs_result) / len(fcfs_result)

print("\nExecution Sequence:")
for i in fcfs_result:
    print(i[0], end=" ")

print("\n\nAverage TAT =", fcfs_avg_tat)
print("Average WT =", fcfs_avg_wt)

print("\n\n== COMPARISON ==\n")

print("SJF Average TAT :", sjf_avg_tat)
print("SJF Average WT :", sjf_avg_wt)

print()

print("FCFS Average TAT :", fcfs_avg_tat)
print("FCFS Average WT :", fcfs_avg_wt)

print()

if sjf_avg_tat < fcfs_avg_tat:
    print("SJF has lower Average Turnaround Time.")
else:
    print("FCFS has lower Average Turnaround Time.")

```

```

if sjf_avg_wt < fcfs_avg_wt:
    print("SJF has lower Average Waiting Time.")
else:
    print("FCFS has lower Average Waiting Time.")

print()

if sjf_avg_tat < fcfs_avg_tat and sjf_avg_wt < fcfs_avg_wt:
    print("Result: SJF is Better than FCFS.")
else:
    print("Result: FCFS is Better than SJF.")

```

Sample input Output

```

PS C:\Program Files\Microsoft VS Code> & C:\Users\lab\AppData\
hon.exe c:/Users/lab/Downloads/sjf.py
File "c:\Users\lab\Downloads\sjf.py", line 15
    remaining processes.copy()
               ^^^^^^^^^^^
SyntaxError: invalid syntax
PS C:\Program Files\Microsoft VS Code> & C:\Users\lab\AppData\
hon.exe c:/Users/lab/Downloads/sjf.py
== SJF ==

Process AT      BT      CT      TAT      WT
P4      1        3        4        3        0
P1      3        3        7        4        1
P5      6        2        9        3        1
P3      5        4       13        8        4
P2      2        5       18       16       11

Execution Sequence:
P4 P1 P5 P3 P2

Average TAT = 6.8
Average WT = 3.4

```

=== FCFS ===

Process	AT	BT	CT	TAT	WT
P4	1	3	4	3	0
P2	2	5	9	7	2
P1	3	3	12	9	6
P3	5	4	16	11	7
P5	6	2	18	12	10

Execution Sequence:
P4 P2 P1 P3 P5

Average TAT = 8.4
Average WT = 5.0

== COMPARISON ==

SJF Average TAT : 6.8
SJF Average WT : 3.4

FCFS Average TAT : 8.4
FCFS Average WT : 5.0

SJF has lower Average Turnaround Time.
SJF has lower Average Waiting Time.

Result: SJF is Better than FCFS.

hon.exe c:/Users/lab/Downloads/sjf.py

== SJF ==

Process	AT	BT	CT	TAT	WT
P4	1	3	4	3	0
P1	3	3	7	4	1
P5	6	2	9	3	1
P3	5	4	13	8	4
P2	2	5	18	16	11

Execution Sequence:
P4 P1 P5 P3 P2

Average TAT = 6.8
Average WT = 3.4

=== FCFS ===

Process	AT	BT	CT	TAT	WT
P4	1	3	4	3	0
P2	2	5	9	7	2
P1	3	3	12	9	6
P3	5	4	16	11	7
P5	6	2	18	12	10

Execution Sequence:
P4 P2 P1 P3 P5

Average TAT = 8.4
Average WT = 5.0

== COMPARISON ==

SJF Average TAT : 6.8
SJF Average WT : 3.4

FCFS Average TAT : 8.4
FCFS Average WT : 5.0

SJF has lower Average Turnaround Time.
SJF has lower Average Waiting Time.

Discussion The program implements and compares two CPU scheduling algorithms: **Shortest Job First (SJF)** and **First Come First Serve (FCFS)**. The processes are defined with their Process ID (P1, P2, etc.), Arrival Time (AT), and Burst Time (BT).

In the **SJF algorithm**, the scheduler checks which processes have already arrived and selects the process with the shortest burst time. If no process is available at the current time, the CPU time is advanced to the arrival time of the next process. This approach allows shorter jobs to be executed first and can reduce the average waiting time and turnaround time.

In the **FCFS algorithm**, processes are arranged according to their arrival time. The process that arrives first is executed first, regardless of its burst time. This algorithm is simple and easy to implement, but a long process arriving earlier can make shorter processes wait for a longer period. For each process, the program calculates:

- **Completion Time (CT)** = Time when the process finishes execution.
- **Turnaround Time (TAT)** = $CT - AT$.
- **Waiting Time (WT)** = $TAT - BT$.

For the given input, the SJF algorithm produces a shorter execution sequence based on burst time among the available processes. The FCFS algorithm follows the arrival order of the processes. After calculating all values, the program calculates the **Average Turnaround Time** and **Average Waiting Time** for both algorithms. The comparison section then determines which algorithm provides better performance based on these two metrics. In this particular example, SJF performs better because it achieves lower average waiting time and lower average turnaround time than FCFS.

Conclusion

In conclusion, the program successfully demonstrates and compares the **SJF and FCFS CPU scheduling algorithms**. FCFS is simple and fair according to arrival order, but it may cause longer waiting times when a process with a large burst time arrives earlier.

On the other hand, **SJF selects the shortest available process first**, which generally improves CPU scheduling efficiency and reduces average waiting and turnaround times. Based on the given process data, SJF provides better results than FCFS in terms of both average waiting time and average turnaround time.

Therefore, the experiment shows that **SJF is more efficient than FCFS for the given set of processes**, although SJF may be less suitable when fairness or starvation prevention is the main concern.

GitHub Repository: <https://github.com/sultan36912/Operating-System-Lab..git>